# The SDK: Guide with Examples

Giolit Trading Risk Verifier 1.0.0 — Tutorial 2

## Contents


1. What is in the SDK . . . 1
2. Building against the SDK . . . 2
3. The license token . . . 2
4. Verifying a specification . . . 2
5. Options . . . 3
6. Results . . . 3
7. Example 1: a release gate . . . 4
8. Example 2: the largest safe parameter value . . . 5
9. Example 3: explaining a violating run . . . 5
10. Example 4: headroom and the resilience score . . . 6
11. Example 5: conformance replay of a FIX log . . . 6
12. Example 6: proofs and the evidence bundle . . . 6
13. Other entry points . . . 7
14. Errors, resources and good practice . . . 7


*Tutorial 2 of 3 for users of the Giolit Trading Risk Verifier 1.0. It assumes you have read Tutorial 1 (`../01_verifying_trading_strategies`) or know how to write a specification.*

The command-line tool `trade-verify` is one client of a set of C++ libraries. The SDK gives your own programs the same capabilities: verify a specification, read every verdict, run and proof as data, compute headroom, replay FIX logs, and write signed evidence bundles. Typical uses:

- a **release gate** in your build system that blocks a strategy release on a violated constraint;
- **parameter studies**: the largest clip size, the tightest limit, the parameter values a client may enter;
- **your own reports and dashboards**: verdicts and runs as data, in your formats;
- **integration** with an OMS, a risk system or a strategy-development platform.

---

## 1. What is in the SDK

| Header (`include/giolit/`) | Library | Provides |
|---|---|---|
| `License.hpp` | `libgiolit_license` | `acquireToken`, `licenseStatus`, `activateLicense` |

| Header (`include/giolit/`) | Library | Provides |
|---|---|---|
| `Trade.hpp` | `libgiolit_trade`, `libgiolit_frontend` | `verifySpec`, `addHeadroom`, `optimizeObjective`, `conformLog`, `lowerSpec`, `importedSpecText` and the result types |
| `Report.hpp` | `libgiolit_report` | `writeReport` (the evidence bundle), `describeProof` (a proof in the specification's terms) |
| `Api.hpp` | — | `AccessToken`, `InputError`, `AccessDenied` |

Supporting libraries: `libgiolit_backend` (the engines) and `libgiolit_interface` (the task and result types). Complete example programs are in `share/giolit/sdk-examples/` of the package (`verify_spec`, `headroom`, `conform`, `certify`) and in this guide's `examples/` folder (`release_gate`, `limit_sweep`, `explain_run`).

## 2. Building against the SDK

The SDK is C++17. With the package installed in its default place:

```
P=$HOME/.local/giolit-trade-verifier
c++ -std=c++17 -O1 -I$P/include my_tool.cpp -o my_tool -L$P/lib -Wl,-rpath,$P/lib \
    -lgiolit_trade -lgiolit_report -lgiolit_frontend -lgiolit_backend -lgiolit_license -lgiolit_interface
```

`examples/build.sh [INSTALL_DIR]` builds this guide's examples. With CMake:

```
set(GIOLIT $ENV{HOME}/.local/giolit-trade-verifier)
add_executable(my_tool my_tool.cpp)
target_include_directories(my_tool PRIVATE ${GIOLIT}/include)
target_link_directories(my_tool PRIVATE ${GIOLIT}/lib)
target_link_libraries(my_tool giolit_trade giolit_report giolit_frontend giolit_backend giolit_license
    giolit_interface)
set_target_properties(my_tool PROPERTIES BUILD_RPATH ${GIOLIT}/lib INSTALL_RPATH ${GIOLIT}/lib)
```

## 3. The license token

Every licensed entry point takes a short-lived **access token**, issued only while the installation's trial or purchased license is valid. Acquire it once per program run:

```
#include "giolit/License.hpp"

giolit::TokenGrant g = giolit::acquireToken();
if (!g.granted) {                       // no valid license: g.message says why and how to get one
    std::cerr << g.message << "\n";
    return 4;                           // the same exit code trade-verify uses
}
// g.tier: "trial" or "paid"; g.runsRemaining (-1: unlimited)
```

Each `acquireToken` counts as one run against the license, like one run of `trade-verify`. A missing, forged or expired token makes the entry points throw `giolit::AccessDenied`. `licenseStatus()` reads the license without charging a run (for a status page).

## 4. Verifying a specification

```
#include "giolit/License.hpp"
#include "giolit/Trade.hpp"

giolit::TradeOptions o;
o.seconds = 60;                                                 // the whole time budget
giolit::TradeResult r = giolit::verifySpec(g.token, "desk.gts", o);
```

```
for (const giolit::ConstraintResult& c : r.constraints)
    std::cout << c.name << ": " << giolit::toString(c.verdict) << " (" << c.method << ")\n";
```

`verifySpec` reads the specification (and the code it imports), builds the models, races the engines and returns when every constraint is decided or the budget is spent. It throws `giolit::InputError` (with `.line`) when the specification is not valid GTSL or an import is not supported.

## 5. Options

`TradeOptions` (in `Trade.hpp`):

| Field | Default | Meaning |
|---|---|---|
| `seconds` | 60 | total time budget |
| `raceModels` | true | the slot model (exact runs) and the aggregate model (proofs for any number of orders) in parallel |
| `aggregateOnly` | false | proofs only (no runs) |
| `certificates` | false | keep each proof's invariants (needed for `describeProof` and certificates) |
| `lower` | — | `SpecOptions`, below |

`SpecOptions` (`o.lower`):

| Field | Default | Meaning | CLI equivalent |
|---|---|---|---|
| `only` | "" | verify only this constraint (its cone of influence: smaller and faster) | `--constraint=NAME` |
| `parameters` | {} | override strategy or market parameters: `{"clip", "120"}`, value `"any"` for arbitrary | `-P NAME=VALUE` |
| `slots` | 4 | K, the live orders the exact model tracks | `--slots=K` |
| `controlsOnly` | false | any strategy instead of the rules | `--controls-only` |
| `sanity` | true | the automatic sanity checks | `--no-sanity` |
| `traceState` | false | keep every displayed variable in runs (full runs, slower proofs) | — |

## 6. Results

`TradeResult`:

| Field | Meaning |
|---|---|
| `spec` | the specification's name |

| Field | Meaning |
|---|---|
| `constraints` | one `ConstraintResult` per constraint and `feasible` target, in file order |
| `sanity` | the sanity checks (a check with verdict `Infeasible` failed: a rule never fires, a control never acts, ...) |
| `notes` | what the model assumes (K, imports, notes about the code) |
| `facts` | units: `#tick`, `#money`, `#currency`, `#timeunit`, ... |
| `seconds` | the time used |

`ConstraintResult`:

| Field | Meaning |
|---|---|
| `name`, `text`, `line` | the constraint as written |
| `verdict` | `Proven`, `ProvenBounded`, `Violated`, `Feasible`, `Infeasible`, `Undecided` (`toString` gives the CLI word) |
| `method` | how it was decided (engine and order model) |
| `cause` | for `Undecided`: why |
| `trace` | for `Violated`/`Feasible`: the run; `trace[0]` is the initial state |
| `proof` | for `Proven` (with `certificates`): the invariants |
| headroom fields | after `addHeadroom`: `worst`, `limit`, `margin`, `relative`, `stars`, ... |

A run step (`TraceStep`) has the `event` (`"Fill O1 100 @ 30001"`), the strategy's `actions` (`"NewOrder O2 BUY 100 @ 30000"`, `"risk control max_qty rejected ..."`), the integer `state` and the real-valued `reals` after the step, and the `orders` (`TraceOrder`: id, side, qty, cum, price in ticks or -1 for market, state, venue, symbol). Prices are in ticks and money in money units; `facts` gives the conversion.

## 7. Example 1: a release gate

`examples/release_gate.cpp` verifies several specifications and decides whether a release may go out: exit 1 on a violation (with the run), 2 when something stayed undecided, 0 when everything is proven.

```
for (int i = 1; i < argc; ++i) {
    giolit::TradeOptions options;
    options.seconds = 120;
    giolit::TradeResult r = giolit::verifySpec(grant.token, argv[i], options);
    for (const giolit::ConstraintResult& c : r.constraints) {
        std::cout << "  " << giolit::toString(c.verdict) << "  " << c.name << "\n";
        if (c.verdict == giolit::Verdict::Violated) {
            ++violated;
            for (size_t k = 1; k < c.trace.size(); ++k) {          // trace[0] is the initial state
                std::cout << "      " << k << "  " << c.trace[k].event << "\n";
                for (const std::string& a : c.trace[k].actions) std::cout << "          -> " << a << "\n";
            }
        } else if (c.verdict == giolit::Verdict::Undecided || c.verdict == giolit::Verdict::ProvenBounded)
     {
            ++open;
        }
    }
    for (const giolit::ConstraintResult& s : r.sanity)
        if (s.verdict == giolit::Verdict::Infeasible) std::cout << "  warning: " << s.text << ": never\n";
}
return violated ? 1 : open ? 2 : 0;
```

Output on the package examples O2 (fixed) and O1 (buggy):

```
$ ./release_gate 02_position_limit_fixed.gts 01_position_limit.gts
== PositionLimitFixed (02_position_limit_fixed.gts, 0.13 s)
  PROVEN  position_limit
== PositionLimit (01_position_limit.gts, 0.19 s)
  VIOLATED  position_limit
      1  MarketUpdate bid=20013 ask=20001 last=20003 volume=5000
           -> NewOrder O1 BUY 100 @ 20013
      2  Ack O1
      3  MarketUpdate bid=20013 ask=24388 last=39999 volume=5000
           -> NewOrder O2 BUY 100 @ 20013
      ...
GATE FAILED: 1 violated, 0 undecided
```

The runs `verifySpec` returns are the runs as found; `trade-verify` additionally shortens them before printing (the CLI's 6-event run of this example), which is why the SDK's run can be longer.

## 8. Example 2: the largest safe parameter value

`examples/limit_sweep.cpp` answers questions such as *"what is the largest clip for which the pair can never be more than 300 out of balance?"* by bisection: every probe overrides the parameter and verifies the one constraint.

```
giolit::Verdict probe(const giolit::AccessToken& tok, const char* spec, const std::string& param, long long
      value,
                      const std::string& constraint) {
    giolit::TradeOptions o;
    o.seconds = 30;
    o.lower.parameters[param] = std::to_string(value);   // like -P clip=120
    o.lower.only = constraint;                           // like --constraint=pair_imbalance
    o.lower.sanity = false;
    giolit::TradeResult r = giolit::verifySpec(tok, spec, o);
    for (const giolit::ConstraintResult& c : r.constraints)
        if (c.name == constraint) return c.verdict;
    return giolit::Verdict::Undecided;
}
```

On the package example `20_pairs_legs_fixed.gts` (a pairs trade: sell `clip` HDFCBANK, buy 150 ICICIBANK, with `pair_imbalance: always abs(3 * position[HDFCBANK] + 2 * position[ICICIBANK]) <= 300`):

```
$ ./limit_sweep 20_pairs_legs_fixed.gts clip pair_imbalance 50 200
  clip = 125: VIOLATED
  clip = 87: PROVEN
  clip = 106: VIOLATED
  clip = 96: PROVEN
  clip = 101: VIOLATED
  clip = 98: PROVEN
  clip = 99: PROVEN
  clip = 100: PROVEN
largest clip with pair_imbalance PROVEN: 100
```

Eight verifications, 49 seconds in all: with clip 101, a pair whose second leg is rejected by the exchange leaves an imbalance of 303. Each answer is a proof or a run, never an estimate. Treat UNDECIDED probes as unsafe, as the example does.

The same pattern answers other questions: the tightest limit a risk control may use, the smallest `max_order_age` that keeps one order working, or every value of a FIXatdl parameter list.

## 9. Example 3: explaining a violating run

`examples/explain_run.cpp` verifies one constraint with `traceState` on and prints the full state and the order book after every event — what an engineer needs to rebuild the scenario in a back-test:

```
$ ./explain_run 01_position_limit.gts position_limit
position_limit: VIOLATED (counterexample (simulation and guided search) [slot model])

0  initial state
```

```
   market.ask = 30812
   market.bid = 30812
   open_orders = 0
   position = 900
   ...
1  MarketUpdate bid=40000 ask=22346 last=20000 volume=0
   -> NewOrder O1 BUY 100 @ 40000
   open_orders = 1
   open_qty(buy) = 100
   position = 900
   order O1 BUY 100 @ 40000 filled 0 PendingNew
2  MarketUpdate bid=25331 ask=39999 last=20009 volume=4
   -> NewOrder O2 BUY 100 @ 25331
   open_orders = 2
   open_qty(buy) = 200
   ...
```

(In this example the specification states no `market.bid < market.ask` assumption, so the market may be crossed — add the assumption and the runs respect it.) For FIX messages instead of text, use `trade-verify --traces=DIR` or the bundle of `writeReport` (section 12).

## 10. Example 4: headroom and the resilience score

```
giolit::TradeOptions o;
o.seconds = 30;
giolit::TradeResult r = giolit::verifySpec(tok, spec, o);
giolit::addHeadroom(tok, spec, o, r, 60);              // 60 s for all limits together
for (auto& c : r.constraints) {
    if (!c.hasHeadroom) continue;
    std::cout << c.name << ": " << c.bounded << (c.upper ? " <= " : " >= ") << c.worst
              << " on every run (limit " << c.limitText << " = " << c.limit << ")"
              << (c.worstExact ? ", reached by a run" : "") << "; " << c.stars << " star(s)\n";
}
std::cout << "Giolit Risk Resilience Score: " << r.resilienceStars << " of 5\n";
```

For every proven `always x <= L` (or `>=`), `addHeadroom` finds the worst value `x` takes on any run by tightening `L` and proving again. `relative` is the unused share of the limit; `stars` rates it (5 from 50 % unused, 1 when a run reaches the limit). Shipped as `sdk-examples/headroom.cpp`.

## 11. Example 5: conformance replay of a FIX log

```
giolit::ConformResult c = giolit::conformLog(tok, "desk.gts", "dropcopy.fix", giolit::SpecOptions{});
for (auto& i : c.issues) {
    if (i.kind == giolit::ConformIssue::Kind::Note) continue;
    std::cout << giolit::toString(i.kind) << " at message " << i.message << " (line " << i.line << "): " <<
      i.why << "\n";
}
std::cout << c.replayed << " of " << c.messages << " messages replayed\n";
```

Issue kinds: `IMPOSSIBLE` (the specification says this cannot happen: an assumption is wrong), `UNEXPECTED` (an order the rules would not send), `MISSING` (an order the rules would send that the log does not show), `CONTROLS` (an order the controls reject reached the venue), `VIOLATION` (the observed run breaks a constraint), `UNMAPPED`. `c.states` has the model's state after every message — useful for a reconciliation view. Shipped as `sdk-examples/conform.cpp`; a nightly job over drop copies is a few lines around it.

## 12. Example 6: proofs and the evidence bundle

```
#include "giolit/Report.hpp"

giolit::TradeOptions o;
o.seconds = 60;
o.certificates = true;                                  // keep the proofs
giolit::TradeResult r = giolit::verifySpec(tok, spec, o);
giolit::addHeadroom(tok, spec, o, r, 60);
```

```
for (auto& c : r.constraints) {
    giolit::TradeProof p = giolit::describeProof(tok, c, r.facts);
    for (auto& rel : p.relations) std::cout << "    " << rel << "\n";   // e.g. open_qty(buy) + position <=
      1000
}
giolit::ReportRequest rq;
rq.specFile = spec;
rq.outDir = "bundle";
rq.issuedTo = "Example Broking Pvt Ltd";
rq.commandLine = {"certify", spec, "bundle"};
rq.engineVersion = "1.0.0";
giolit::ReportOutcome ro = giolit::writeReport(tok, r, rq);
// ro.verdict: VERIFIED / VIOLATIONS / INCOMPLETE; ro.pdfPath; ro.signedByLicensee
```

`describeProof` turns a proof into relations over the trading state, in the specification's names: the inductive invariant that makes the constraint hold. `writeReport` writes the same bundle as `trade-verify --report`: the PDF, the signed statement, the proofs, every run as text/JSON/FIX/CSV, and `verify_certificate.py`. The statement is signed with your signing credential when a purchased license is active. Shipped as `sdk-examples/certify.cpp`.

## 13. Other entry points

| Function | Use |
|---|---|
| `optimizeObjective(tok, spec, o, r, seconds)` | the best and worst value of the specification's `objective` over all runs: each a value a run reaches and a proved bound (`r.objective`) |
| `importedSpecText(tok, spec)` | the GTSL your binding generates from your code (`--import-only`): show it in a review tool |
| `lowerSpec(tok, spec, options)` | the transition system itself (`vtask::Task`), for tooling that inspects the model |
| `licenseStatus()` | runs used and allowed, expiry, licensee — without charging a run |
| `activateLicense(file, &buyer)` | apply a purchased license from your own installer |

## 14. Errors, resources and good practice

- **Exceptions.** `InputError` (with `.line`) for a specification or import that cannot be read; `AccessDenied` for a missing or expired token. Nothing else is thrown for normal results: a violated constraint is a result, not an error.
- **Processes.** `verifySpec` runs its engines in child processes (up to six at a time, fewer on small machines) and stops them when a constraint is settled or the budget ends. Call it from one thread at a time; run several verifications one after another, not in parallel.
- **Memory.** A verification adapts to the memory it may use (the machine's, or a container or cgroup limit): it starts fewer engines and stops the largest one before the limit. Give it at least 4 GB; 8 GB is comfortable for strategies with several venues or symbols.
- **Time.** `seconds` is a budget, not an estimate: most controls-only proofs take well under a second, strategies with real-number state may take minutes. Verify one constraint (`lower.only`) when you need one answer fast.
- **Determinism.** Proofs and violations are facts and do not change between runs. Which run is shown for a violation, and whether a hard constraint is decided within a short budget, can vary; give release gates enough time.
- **Units.** Prices in runs are in ticks and money in money units; use `r.facts` to convert.

---

Giolit Labs, Thrissur, Kerala, India — www.giolit.com — business.giolitlabs@gmail.com